

Lecturer: Yetmwork Tesfaye

MSc. in Computer Science and Engineering

College of Computing and Informatics

Department of Computer Science

Chapter 8: Fault Tolerance

Objectives

At the end of this Chapter Students be able to know:-

- Basic Concepts
- Failure Models
- Process Resilience
- Distributed Commit
- Recovery

Introduction

- a major difference between distributed systems and single machine systems is that with the former, partial failure is possible, i.e., when one component in a distributed system fails, such a failure may affect some components while others will continue to function properly.
- an important goal of distributed systems design is to construct a system that can **automatically recover from partial failure**.
- it should tolerate faults and continue to operate to some extent.

Basic concepts



- Fault tolerance is strongly related to dependable systems.
- **dependability** covers the following terms:
 - **Availability**
 - refers to the probability that the system is operating correctly at any given time; defined in terms of an instant in time.
 - **Reliability**
 - a property that a system **can run continuously without failure**; defined in terms of a time interval.



Basic concepts

- **Safety**

- refers to the situation that even when a **system temporarily fails to operate correctly**, nothing catastrophic happens.

- **Maintainability**

- how easily a failed system can be repaired.

Cont...



- Dependable systems are also required to provide a high degree of **security**. A system is said to **fail** when it cannot meet its promises; for instance failing to provide it users one or more of the services it promises.
- An **error** is a part of a system's state that may lead to a **failure**; e.g., damaged packets in communication.
- The cause of an error is called a **fault**. Clearly, finding out what caused an error is important. For example, a wrong or bad transmission medium may easily cause packets to be damaged. In this case, it is relatively easy to remove the fault.



Cont....

- Building dependable systems closely relates to **controlling faults**. There is a difference between **preventing**, **removing**, and **forecasting** faults.
- The most important issue is **fault tolerance**, meaning that a system can provide its services even in the presence of **faults**. For example, by applying error correcting codes for transmitting packets, it is possible to **tolerate**, to a certain extent, relatively poor transmission lines and reducing the probability that an error (a damaged packet) may lead to a failure.

Faults are Classified into Three



- **Transient**

- Occurs once and then disappears; if the operation is repeated, the fault goes away;

- **Intermittent**

- It occurs, then vanishes on its own accord, then reappears, ...;

- **Permanent**

- One that continues to exist until the faulty component is repaired; e.g, disk head crash, software bug

Failure Modes(Types)



A system that fails is not adequately providing the services it was designed for to get a better grasp on how serious a failure actually is, several classification schemes have been developed.

- **Crash failure:** a server halts, but was working correctly until it halts.
- **Omission failure:** a server fails to respond to incoming requests.
 - **Receive omission:** a server fails to receive incoming messages; e.g., may be no thread is listening.
 - **Send omission:** a server fails to send messages.

Cont...

- **Timing failure:** a server's response lies outside the specified time interval; e.g., may be it is too fast over flooding the receiver or too slow.
- **Response Failure:** A server's response is incorrect
 - **Value Failure:** The value of the response is wrong.
 - **State transition failure:** The server deviates from the correct flow of control.
- **Arbitrary Failure:** A server may produce arbitrary responses at arbitrary times.

Failure Masking

- If a system is to be fault tolerant, the best it can do is to try to hide the occurrence of failures from other processes - masking
- The key technique for masking faults is **redundancy**.
- Three kinds of redundancy are possible:
 - **Information redundancy**; add extra bits or redundant data to the transmitted or stored information to allow recovery from garbled bits (error correction).

Cont...

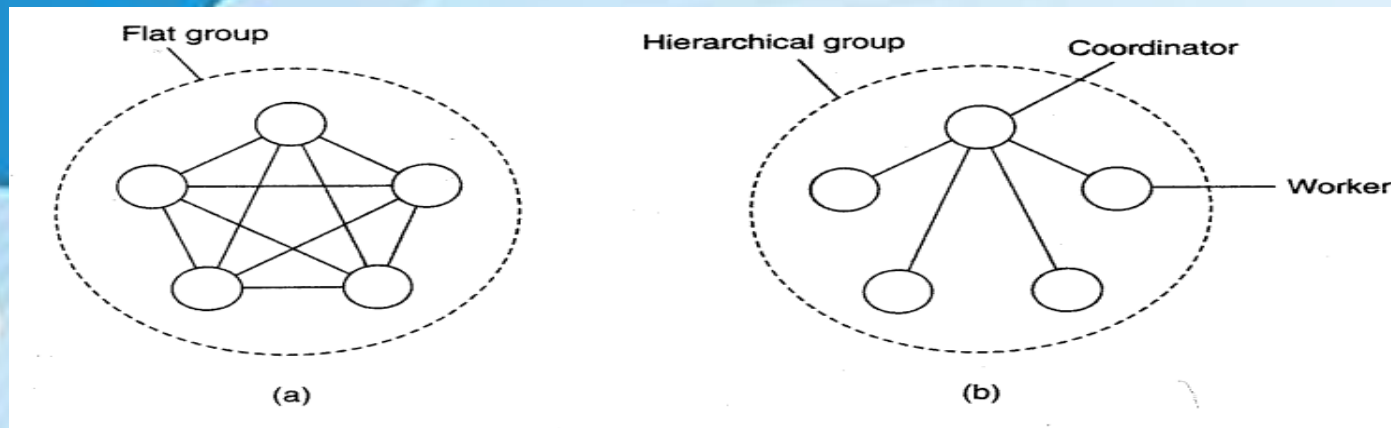
- **Time redundancy:** an action is performed more than once if needed; e.g., retry or redo an aborted transaction; useful for transient and intermittent faults.
- **Physical redundancy:** add (replicate) extra equipment (hardware) or processes (software) to the system.

Process Resilience

- How can fault tolerance be achieved in distributed systems?
- One method is **protection against process failures** by replicating processes into **groups**
- Design issues of process groups
 - The key approach to tolerating a faulty process is to organize several identical processes into a group.
 - The key property that all groups have is that when a message is sent to the group itself, all members of the group receive it.
 - In this way, if one process in a group fails, hopefully some other process can take over for it.

Cont...

- Process groups may be dynamic.
 - New groups can be created and old groups can be destroyed.
 - A process can join a group or leave one during system operation.
 - A process can be a member of several groups at the same time.
- Hence group management and membership mechanisms are required.



a) Flat group

b) hierarchical group

- Groups may be flat(all processes are equal) or hierarchical(a coordinator and several workers).
- The **flat group**: has no single point of failure, but decision making is more complicated(voting may be required for decision making)
For example, to decide anything, a vote often has to be taken, incurring some delay and overhead.
- The hierarchical group has the opposite properties.
 - it has a single point of failure
 - group membership may be handled
 - through a group server where all requests (joining, leaving, ...) are sent;
- in a distributed way (membership is multicasted)

Failure Masking and Replication

- Process groups are part of the solution for building fault-tolerant systems.
- How to replicate processes so that they can form groups?
- there are two ways :
 - **Primary-based protocols:** for fault tolerance, primary-backup protocol is used; organize processes hierarchically and let the primary coordinate all writes; if the primary crashes, the backups hold an election.
 - **Replicated-write protocols:** in the form of active replication or by means of quorum-based protocols; processes are organized as flat groups.

Cont...



- Another important issue is how much replication is needed.
- A system is said to be **k fault tolerant** if it can survive faults in K components and still meets its specifications.
 - if the processes **fail silently**, then having $k+1$ replicas is enough; if k of them fail, the remaining one can function.
 - if processes exhibit **Byzantine failure**, $2k+1$ replicas are required; if the k processes generate the same reply (wrong), the $k+1$ will also produce the same answer (correct); but which of the two is correct is difficult to ascertain; unless we believe the majority.



Reliable Client-Server Communication

- Fault tolerance in distributed systems concentrates on faulty **processes**
- but communication failures also have to be considered
- a communication channel may exhibit failures in the form of
 - Crash
 - Omission
 - Timing
 - Arbitrary (duplicate messages as a result of buffering at nodes and the sender retransmitting).

Point-to-Point Communication

- Reliable transport protocols such as TCP can be used that mask most communication failures such as omissions (lost messages) using acknowledgements and retransmissions.
- However, crash failures are often not masked: e.g., a broken TCP connection; in which case the system may probably setup a new connection automatically.

RPC or RMI Semantics in the Presence of Failures

- The goal of RPC or RMI is to hide communication by making remote procedure calls look like local ones; may be difficult to mask remote calls when there are failures.
- Five different classes of failures can occur in RPC or RMI systems, each requiring a different solution:
 - The client is unable to locate the sever
 - The client's request to the server is lost
 - The server receives the request and crashes
 - The reply message from the server to the client is lost
 - The client crashes after sending a request

Cont...

1. The client is unable to locate the sever

- may be the client is unable to locate a suitable server or the server is down
- or an interface at the server has been changed making the client's stub obsolete (it was not used for a long time) and the binder failing to match it with the server
- one solution: let the client raise an exception and have exception handlers; inserted by the programmer
- examples are exceptions in Java or signal handlers in C But not every language may provide exceptions or signals

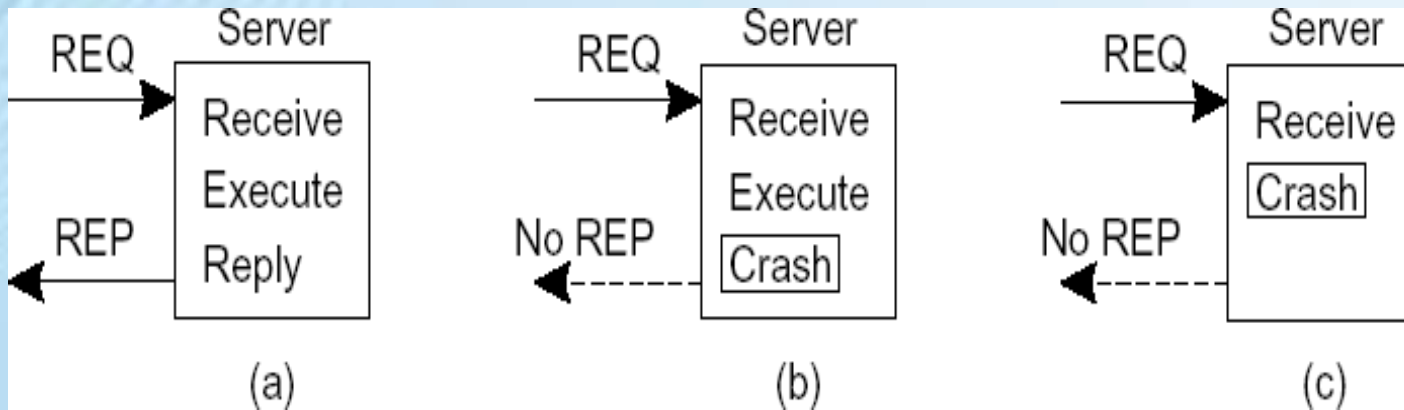
2. The client's request to the server is lost

- let the OS or the client stub start a timer when sending a request; if the timer expires **before a reply or acknowledgement comes back**, the message is retransmitted

Cont...

3. The server receives the request and crashes

- the failure can happen before or after execution



a server in client-server communication; (a) normal case, (b) crash after execution, c) crash before execution

- in (b), let the system report failure back to the client (raise an exception)
- in (c) retransmit the request
- but the client's OS doesn't know which is which; only a timer has expired

Cont...

- Three possible solutions
 - The client tries the operation again after the server has restarted (or rebooted) or rebinds to a new server, called **at least once semantics**; keep on trying until a reply is received; it guarantees that the RPC has been carried out at least one time, but **possibly more**
 - The client gives up after the first attempt and reports error, called **at most once semantics**; it guarantees that the RPC has been carried out at most one time, but **possibly none at all**
 - No guaranty at all (**0-N times**)
- None of the above is the right solution; what is required is **exactly once semantics**; but difficult to achieve

Cont...

4. The reply message from the server to the client is lost

- The client sets a timer and resends
- There is a risk that the request is **sent more than once**; may be the server was slow.
- However, if the operation is **idempotent** (an operation such as requesting a read that can be repeated without any harm), it does not create any problem; but what if the request is to transfer money from one account to another?
- Let the client assign a **sequence number to each request so that the server knows it is a duplicated one**; but needs the server to maintain information on each client.
- In addition, we can have **a bit in the message header** that is used to **distinguish initial requests from retransmissions** so as to take more care on retransmissions.

Cont...

5. The client crashes after sending a request

- In this case a computation may be done without a parent waiting for the result; such **unwanted computations** are called **orphans**
- Orphans
 - waste CPU cycles
 - can lock valuable resources such as files
 - can cause confusion if the client reboots and does the RPC again and the reply from the orphan comes back immediately afterward
- Proposed solutions
 - **Extermination**: let the client stub maintain **a log before sending** an RPC message; after a reboot, **inspect the log and kill the orphan**;
 - some problems
 - The expense of writing a log for every RPC

Cont...



- **reincarnation**: divide time into sequentially numbered epochs; when a client reboots, it broadcasts a message to all machines declaring the start of a new epoch; when such a broadcast comes in, all remote computations on behalf of that client are killed.
- **gentle reincarnation**: when an epoch broadcast comes in, each machine checks to see if it has remote computations, and if so, tries to locate their owner; a computation is killed only if the owner cannot be found.



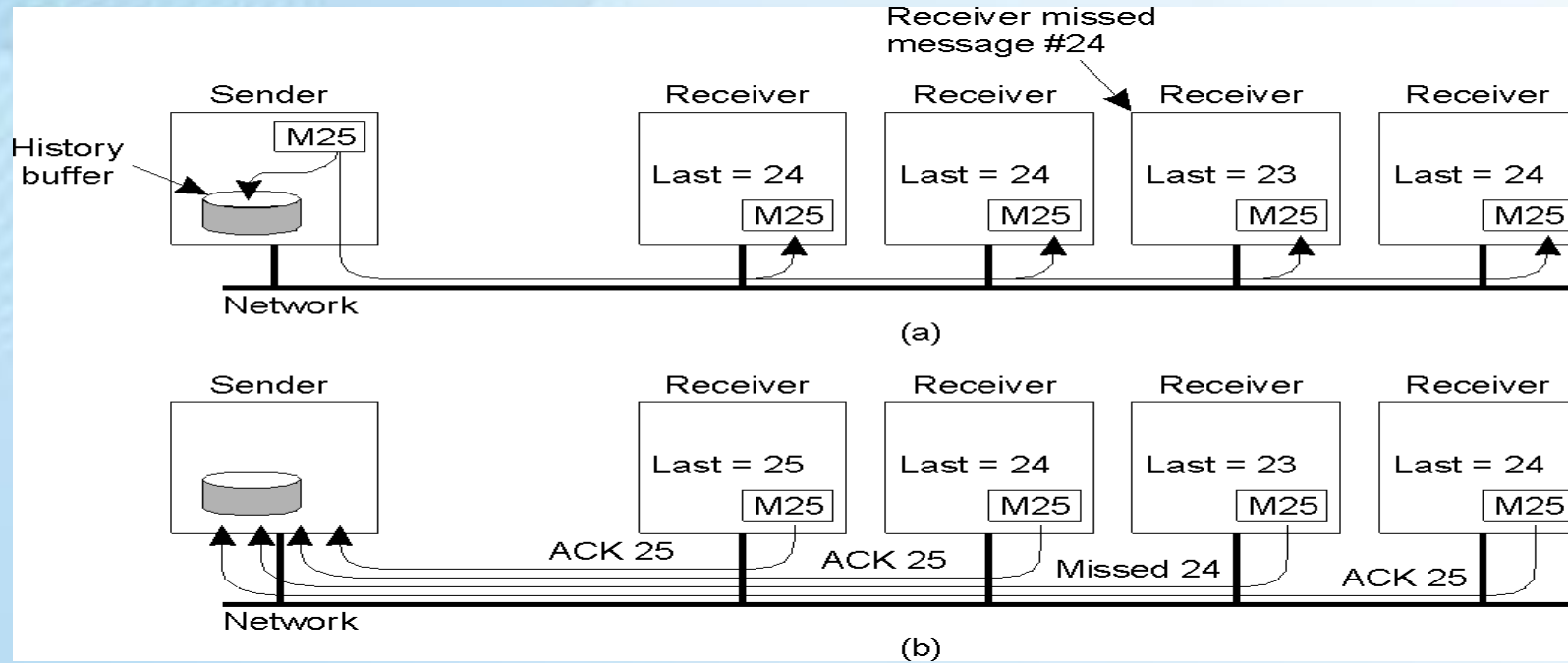
Cont...

- **expiration**: each RPC is given a standard amount of time T , to do the job; if it cannot finish, it must explicitly ask for another quantum; if after a crash the client waits a time T before rebooting, all orphans are killed; the problem is how to choose a reasonable value of T , since RPCs have differing needs.
- none of the solutions is satisfactory; still locked files or database records will remain locked if an orphan is killed, etc.

Reliable Group Communication

- How to reliably deliver messages to a process group (multicasting) - replicating
 - **Basic Reliable-Multicasting Schemes**
 - reliable multicasting means a **message sent to a process group should be delivered to each member of that group.**
 - transport protocols **do not offer** reliable communication to a collection of processes
 - problems:
 - what happens if a process joins a group during communication?
 - what happens if a (sending) process crashes during communication?
 - what if there are faulty processes?
 - a weaker solution assuming that all receivers are known and that none will fail is for the sending process to assign a sequence number to each message and to buffer all messages so that lost ones can be retransmitted

Cont...



a simple solution to reliable multicasting when all receivers are Known and are assumed not to fail;
(a) message transmission,
(b) reporting feedback

Atomic Multicast

- **Atomic Multicast**

- how to achieve **reliable multicasting** in the presence of process failures?
- for example, in a replicated database, how to handle update operations when a replica crashes during update operations?
- the **atomic multicast problem**: to guarantee that a message is delivered to either all processes or none at all **and** that messages are delivered in the **same order to all processes**.

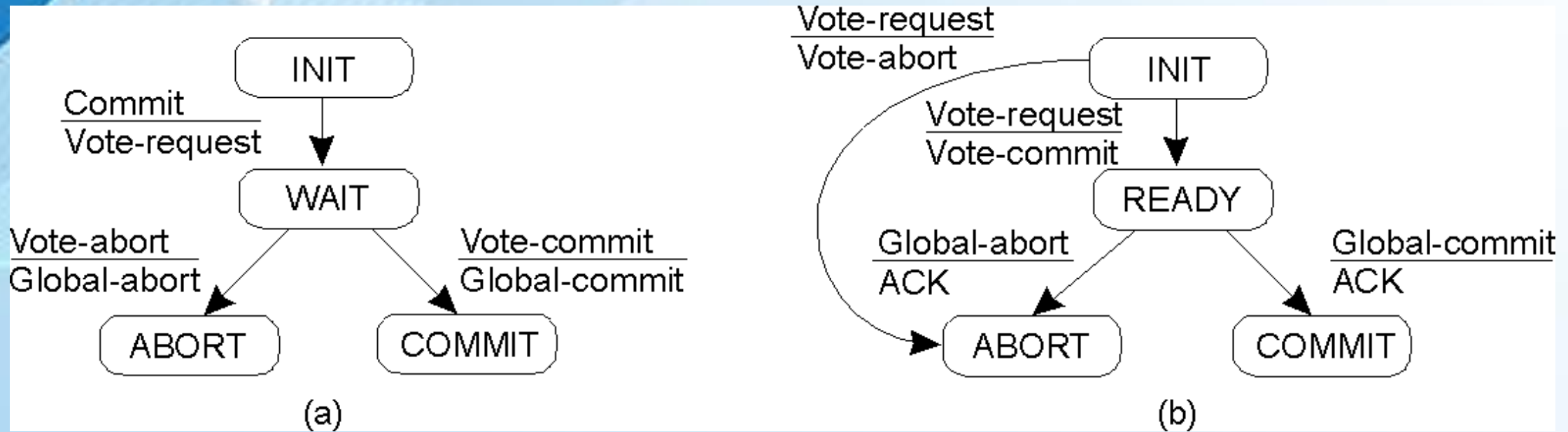
Distributed Commit

- Atomic multicasting is an example of the more generalized problem known as **distributed commit**
- in atomic multicasting, the **operation is delivery of a message**
- but the **distributed commit** problem involves having an **operation** being performed by each member of a process group, or none at all
- there are three protocols: **one-phase commit**, **two-phase commit**, and **three-phase commit**
- **One-Phase Commit Protocol**
 - a **coordinator** tells all other processes, called **participants**, whether or not to (locally) perform an operation
 - drawback: **if one of the participants cannot perform the operation**, there is no way to tell the coordinator; for example due to violation of concurrency control constraints in distributed transactions

Two-Phase Commit Protocol (2PC)

- It has two phases: **voting phase** and **decision phase**, each involving two steps
- Voting phase
 - the coordinator sends a VOTE_REQUEST message to all participants
 - each participant then sends a VOTE_COMMIT or VOTE_ABORT message depending on its local situation
- Decision phase
 - The coordinator **collects all votes**; if all vote to commit the transaction, it sends a GLOBAL_COMMIT message; if at least one participant sends VOTE_ABORT, it sends a GLOBAL_ABORT message
 - Each participant that **voted for a commit waits for the final reaction** of the coordinator and commits or aborts

Cont...



- a) The finite state machine for the coordinator in 2PC
- b) The finite state machine for a participant

Cont...

- Problems may occur in the event of failures
- The **coordinator and participants** have states in which they block waiting for messages: INIT, READY, WAIT
- When a process crashes, other **processes may wait indefinitely**
- Hence, **timeout** mechanisms are required
- A participant waiting in its **INIT** state for VOTE_REQUEST from the coordinator aborts and sends VOTE_ABORT if it does not receive a vote request after some time.
- The coordinator blocking in state **WAIT** aborts and sends GLOBAL_ABORT if all votes have not been collected on time.
- a participant P waiting in its **READY** state waiting for the global vote cannot abort; instead it must find out which message the coordinator actually sent
 - by blocking until the coordinator recovers
 - or requesting another participant, say Q

Cont...

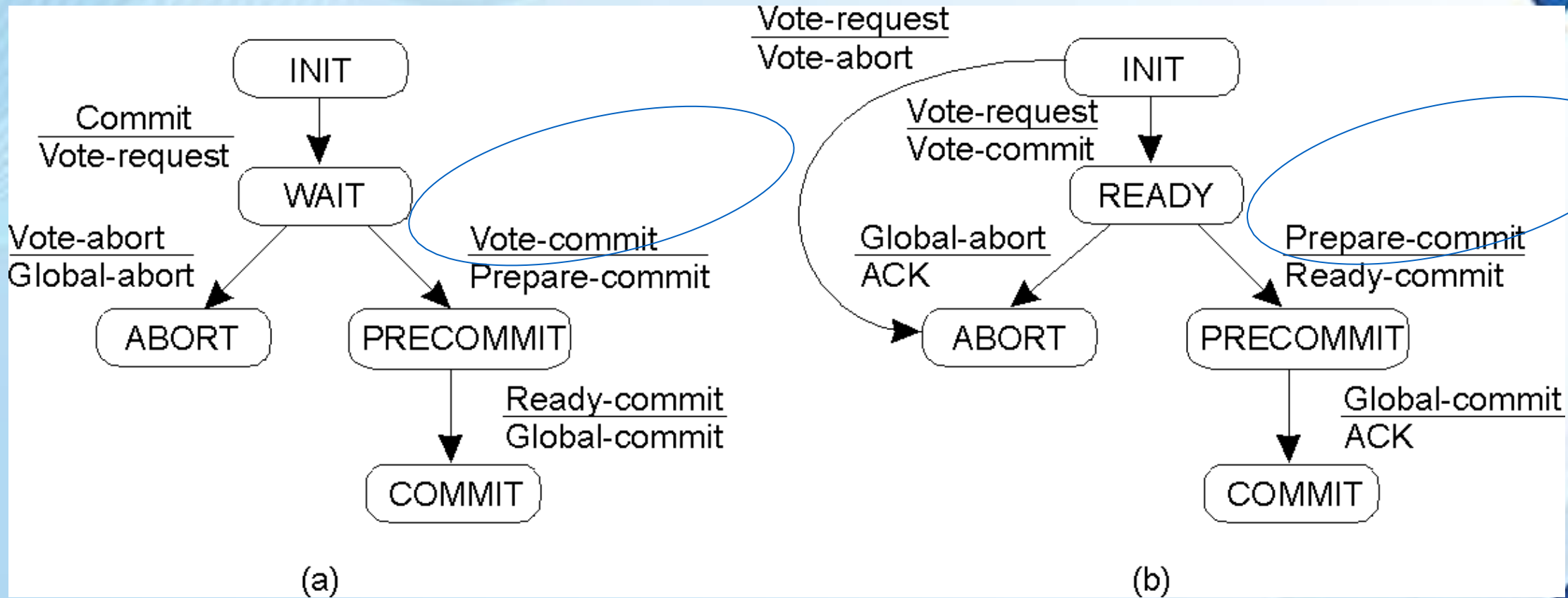
State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Actions taken by a participant P when residing in state READY and having contacted another participant Q

Three-Phase Commit Protocol (3PC)

- The problem with 2PC is that, if the coordinator crashes, participants will need to **block until the coordinator recovers**
- 3PC avoids blocking processes in the presence of crashes.
- The states of the coordinator and each participant satisfy the following two conditions
 - there is no single state from which it is possible to make a transition directly to either COMMIT or an ABORT state
 - there is no state in which it is not possible to make a final decision, and from which a transition to a COMMIT state can be made.

Three-Phase Commit Protocol (3PC)



- a) finite state machine for the coordinator in 3PC
- b) finite state machine for a participant

Recovery

- Fundamental to fault tolerance is recovery from an error
- Error recovery means to replace an erroneous state with an error-free state
- Two forms of error recovery: **backward recovery** and **forward recovery**
- **Backward Recovery**
 - bring the system from its present erroneous state back into a previously correct state
 - for this, the system's state **must be recorded from time to time**; each time a state is recorded, a **checkpoint** is said to be made
 - e.g., retransmitting lost or damaged packets in the implementation of reliable communication

Recovery

- disadvantages:
 - checkpointing and restoring a process to its previous state are costly and performance bottlenecks
 - no guarantee can be given that **the error will not recur**, which may take an application into **a loop of recovery**
 - some actions may be **irreversible**; e.g., deleting a file, handing over cash to a customer

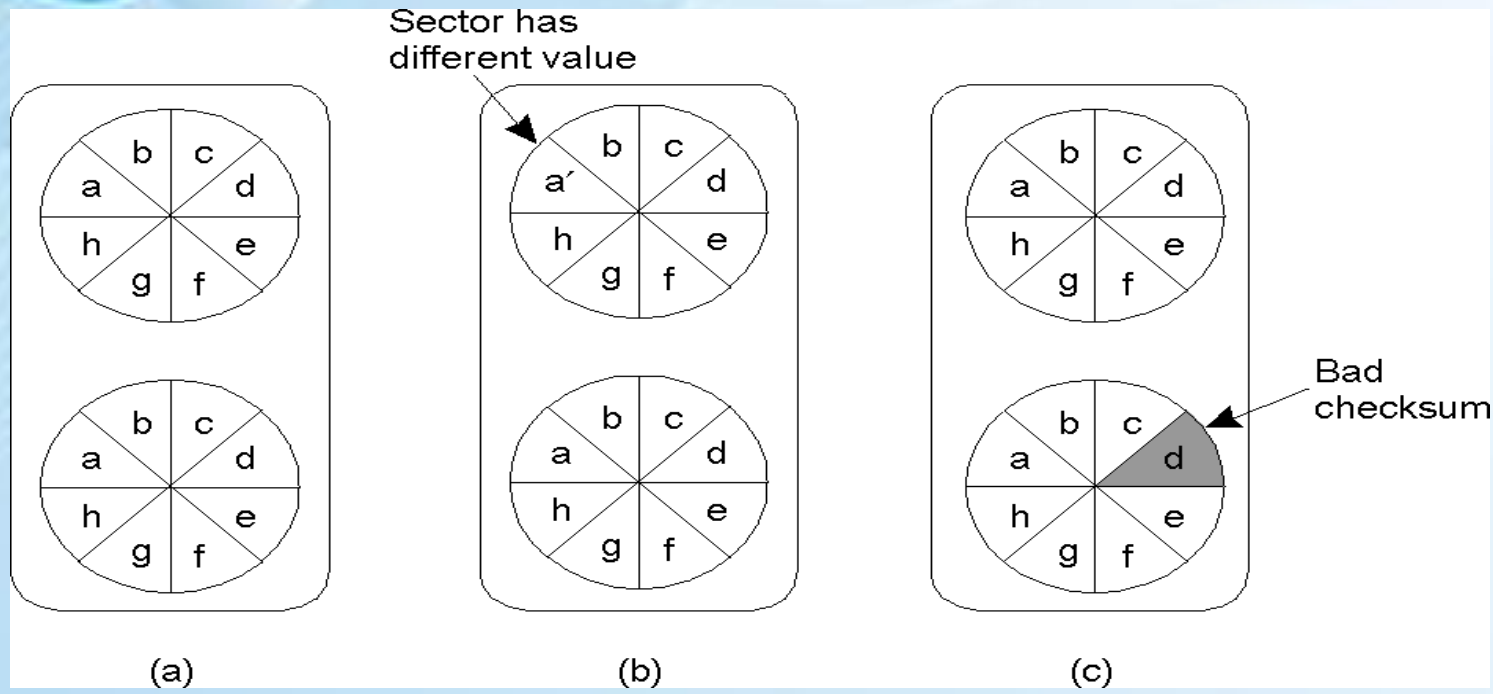
Recovery

- Forward Recovery
 - bring the system from its present erroneous state to a correct new state from which it can continue to execute.
 - it has to be known in advance which errors may occur so as to correct those errors
 - e.g., erasure correction (or simply error correction) where a lost or damaged packet is constructed from other successfully delivered packets.

Stable Storage

- Process states must be **safely** saved for recovery
- how to design a storage system that can survive most problems
- use a pair of disks
 - each block on drive 2 is an exact copy of the corresponding block on drive 1
 - when a block is updated, first the block on drive 1 is updated and verified, then the same block on drive 2 is done.

Stable Storage

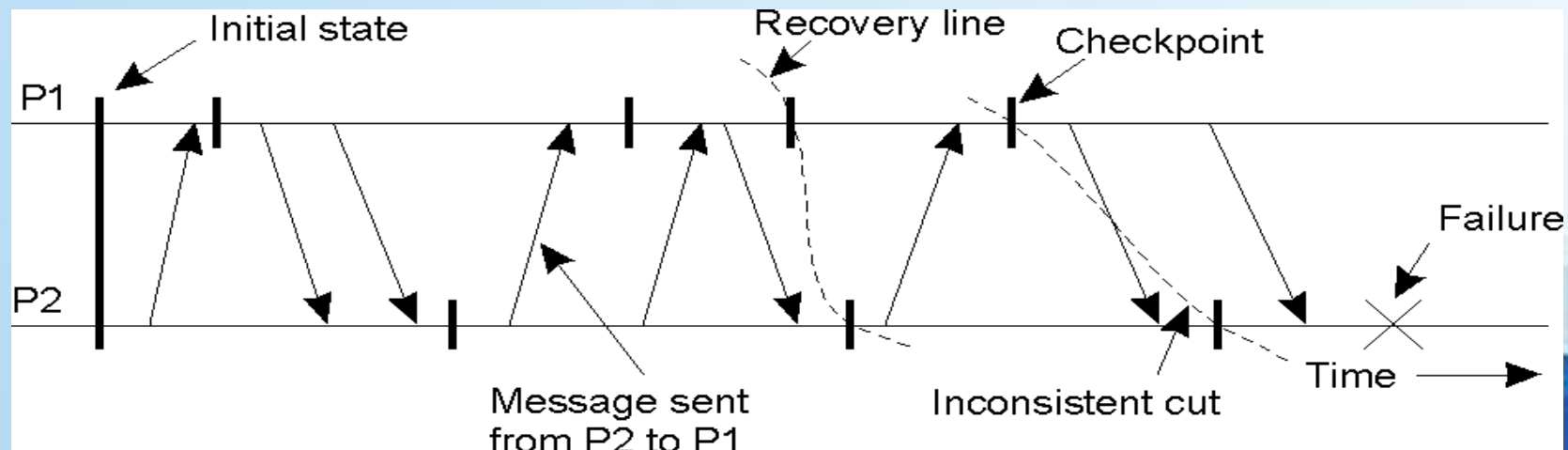


- a) Stable Storage
- b) Crash after drive 1 is updated; upon recovery, the disk can be compared block for block; if corresponding blocks differ, the block on drive 1 is assumed to be correct
- c) Bad spot; copy the corresponding block from the other drive

Stable Storage

■ Checkpointing

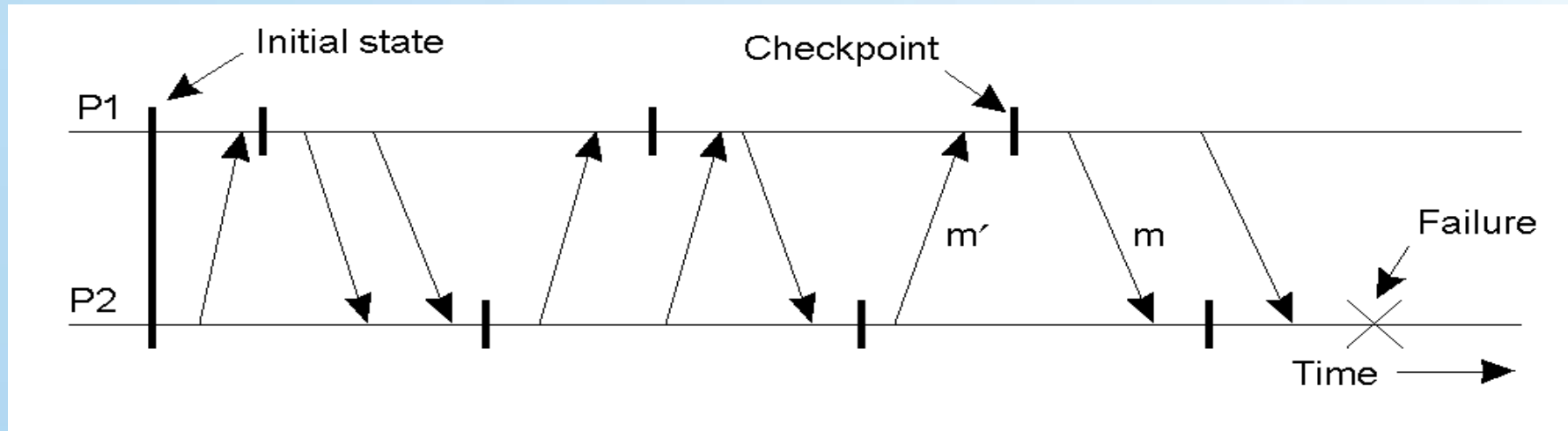
- To recover after a process or system failure requires that we construct a **consistent global** state(also called **distributed snapshot**) from the local states of each process ; i.e., if a process has recorded the receipt of a message, then there should be another process that has recorded the sending of that message
- it is best to recover from the **most recent** distributed snapshot, also referred to as a **recovery line**; it corresponds to the most recent consistent cut



Stable Storage

■ Independent Checkpointing

- discovering a recovery line requires that each process should roll back to its most **recently saved state**; if the local states jointly do not form a distributed snapshot, **further rolling back is necessary**
- this process of cascaded rollback may lead to what is called the **domino effect**, i.e., it may not be possible to find a recovery line except the initial states of processes
- this happens if processes checkpoint independently



the domino effect

Stable Storage

■ Coordinated Checkpointing

- all processes synchronize to jointly write their state to local stable storage
- the **saved state** is automatically **globally consistent**, so that cascaded rollback leading to the domino effect is avoided
- the **distributed** snapshot algorithm can be used to coordinate checkpointing
- or a two-phase blocking protocol can be used (**centralised**)
 - a coordinator multicasts a CHECKPOINT_REQUEST message to all processes
 - in receipt of such a message, each process takes a local snapshot and acknowledges to the coordinator
 - the coordinator then multicasts a CHECKPOINT_DONE message

Thank You!!

